Telomy — Detailed Engineering / Project Report (DPR)

Branch: `telomy/twin3d-sinc-design-ring` · Stack: React Native (Expo) app + FastAPI backend + on-device native modules.

This DPR documents **what the engine computes and how**, the system architecture, every model with its formula and citation, the AI/RAG/SLM stack, the extensibility (science registry), safety, and the multi-user design. It describes the code that is deployed and on the device as of 2026-10-19.

> Sibling project note: the team's repo `mclarenmervin/Telomy` is a **separate stack** — Flutter + Supabase (Postgres, > Auth, Row-Level Security) — and their `healthagent` branch independently arrived at the same AI pattern we use > (retrieval/context → LLM with a deterministic fallback → output guardrails → baseline analysis). The convergence > validates the design; the two apps differ in platform and in how multi-user is done (they use Supabase RLS; our > design below uses a per-user database).

---

1. System architecture

`

■■■

2. Data model (Vault)

Table	Holds	Key
`profile`	DOB, sex, height, weight, goal, flags	single row
`reports` / `results`	lab panels and per-marker values (num/text, unit, ref range, status)	report → many results
`signals`	daily wearable metrics (one value per day × metric)	(day, metric)
`events`	logged alcohol, caffeine, sauna, workouts, supplements, symptoms…	ts, kind
`meals`	analysed meals (kcal/macros/score)	ts
`imaging`	CT/MRI/DEXA/US findings (JSON)	id
`trackers` / `tracker_entries`	talk-to-track items + readings	id
`insights`	engine findings with receipts + review state	stable id (hash)
`chats` / `messages`	Sinc conversations	id
`science_findings`	new science added as data (§8)	id
`twin_projections`	labelled lab projections	(made_on, marker, day)

Markers are defined in `catalog*.py` as `(name, panel, unit, optimal_low, optimal_high, better, aliases, loinc)`;

`better ∈ {low, high, mid}` sets the direction of "good". Synthetic data always carries `is_test_data = 1` and the engine prefers real data (`ORDER BY is_test_data ASC`).

---

3. Biological age (how "blood age" is calculated)

PhenoAge (Levine et al. 2018) — the headline number. From 9 markers + chronological age:

1. Linear predictor

`xb = –19.907 – 0.0336·albumin(g/L) + 0.0095·creatinine(µmol/L) + 0.1953·glucose(mmol/L) + 0.0954·ln(CRP mg/dL) – 0.0120·lymph% + 0.0268·MCV + 0.3306·RDW + 0.00188·ALP + 0.0554·WBC(10■/L) + 0.0804·age`

(unit conversions applied in `engine.phenoage`).

2. 10-year mortality `M = 1 – exp(–exp(xb)·(exp(120·γ)–1)/γ)`, γ = 0.0076927.

3. Phenotypic age `PhenoAge = 141.50225 + ln(–0.00553·ln(1–M)) / 0.090165`.

Inputs are mapped from routine-report marker ids (`_pheno_aliases`: total WBC cells/mm³ → 10³/µL; standard CRP stands

in for hs-CRP). If the latest panel lacks any input, the engine says exactly which (`missing: [...]`) rather than

guessing. A per-report series gives the **trend** (e.g. 22.3 → 20.6 → 19.3 across three panels).

Sensitivity study (`tools/bloodage_study.py`, 50 000 Monte-Carlo combinations within clinically plausible ranges):

RDW explains **51 %** of the spread, then glucose 12 %, creatinine 10 %, MCV 9 % — so RDW (iron/B12/folate status) is

the single biggest lever. Shown as a standardised-regression variance decomposition (R^2 0.99, additive model).

Lab epigenetic clocks (GrimAge, Horvath, DunedinPACE) are shown alongside when present but never substituted for PhenoAge.

---

4. Disease risk (`predict.py`)

Each model is a published equation, computed only from the inputs it has; missing inputs are named, not imputed.

Model	Outcome	Reference
AHA PREVENT (2024)	10- & 30-year total CVD, ASCVD, heart failure, CHD, stroke	Khan SS et al., Circulation 2024
Pooled Cohort Equations	10-year MI/stroke	ACC/AHA 2013
Diabetes risk	5-year T2D	risk-factor model (HbA1c, glucose, BMI, family hx)
FIB-4	advanced liver fibrosis likelihood	Sterling 2006
KFRE	5-year kidney failure	Tangri 2011

Inputs come from `inputs_from_vault()` (latest labs + profile + signal means). `what_if()` recomputes risk under a changed input for the twin.

4b. Trained ML model suite (`research/*`, served via `llx/`)

Beyond the published equations, Telomy ships **~37 trained models** (gradient-boosted trees / clocks), each with a held-out metric and provenance. Full inventory in `docs/research/MODELS_STATUS.md`.

Family	Models	Data & scale	Metric
Blood-panel disease screen	14 (diabetes, HTN, CHD, HF, stroke, CKD, COPD, cancer, liver…)	NHANES ~30k	AUC 0.73–0.89
Lab-free chronic suite	11 (diabetes, CHD, COPD, stroke, depression, arthritis…)	**BRFSS 2011–2023, 5.89M**	AUC 0.70–0.84
Aging / body clocks	blood-age, muscle-age, PhenoAge	NHANES	MAE 8.8 / 9.1 y
Organ clocks ×5	metabolic/hepatic/renal/immune/cardiometabolic	NHANES 49.7k	MAE 11.9–13.9 y (/me/organs)
Nutrition / deficiency	NOVA food-processing, micronutrient	703k / ~10k	acc 0.81 / AUC 0.72
Signal	AFib/rhythm (GBDT, F1 0.783), sleep-stager, fall-detect	CinC-2017 / sleepstage PSG	trained (/me/rhythm)

Serving — the doctor clinic engine (`llx/clinic_engine.py`, `POST /mgmt/clinic/assess` + `/validate`): one patient → one grounded, **clinician-gated** assessment unifying diagnostics (blood-panel models when bloods exist, else the lab-free chronic model + 10-yr mortality + wearable fusion overlay), **evidence-graded treatments** (lifestyle / referral / medication-class-to-discuss / monitoring), **therapies** (centre modalities), and **supplements** (safety-screened). Claims firewall (§12.1 of the master DPR) enforced: A–D grades, "supports/consider/discuss", never "treats"; nothing reaches the member without sign-off. Roadmap + honest per-model data-scale in `docs/research/TOP20_MODELS.md`.

---

5. Correlation / effect engine (`engine.effect_table`, `event_effects`)

Question: does a logged behaviour (alcohol, sauna, late meal, hard workout…) change a next-day outcome (HRV, RHR,

deep sleep, sleep score, sleep hours) in *this person's* data?

Method: one ordinary-least-squares model per outcome:

`outcome(t) ~ 1 + Σ(every event kind on day t−1) + weekend + linear_trend`.

Adjusting for the other events + weekend separates confounds (e.g. sauna on Tue/Sun from hard workouts on Mon/Thu).

For each event coefficient β: standard error from $\sigma^2 \cdot (X'X)^{-1}$, p from the normal approx, Cohen's d = β/σ_y, and the raw (unadjusted) difference kept in the receipts. **Benjamini–Hochberg** controls the false-discovery rate across all tests (q = 0.05); only effects with p ≤ cut **and** |d| ≥ 0.3 and ≥ min_events occurrences are reported.

Calibration (100-person synthetic cohort with planted, mediated truth, `sim/`): 95 % CI coverage, 1.2 % false-positive rate, 100 % sign accuracy. Trackers get the same treatment with Spearman ρ + BH (`trackers.drivers`).

---

6. Sleep (how stages and scores are computed)

Staging, by source (best first): patch ECG (Conv1d+BiGRU on HRV + ECG-derived features) → WHOOP (motion+HR stager, `stager_motion.py`, LOSO κ 0.47 on PhysioNet sleep-accel) → Apple Watch stages → ring firmware → **ring HR estimate**.

The ring-HR estimate (new) gives a coarse 4-stage hypnogram from the 5-minute heart-rate curve + phone motion (deep = HR near nightly trough + low variability, front-loaded; REM = HR lifted + variable + still, not in the first 15 min; de Zambotti 2019 cardiac method), labelled low-confidence and down-weighted vs the patch.

Fusion (`sleepfuse.ts`): one night per sleep period, source priority above, fragments merged (< 90 min gap), uncovered tails kept. **Sleep need** uses NOOP's validated 0.55 recurrence on recent debt + strain.

Sleep score (`wellness-core.sleepScore`): weighted contributors — duration vs personal need (NSF 7–9 h), efficiency (Ohayon ≥ 85 %), latency (≤ 15 min), WASO (≤ 20 min), deep %, REM %, awakenings, and HRV restoration (overnight ln RMSSD vs a robust winsorised personal baseline). Weights renormalise over what's available; estimated stages count less.

Recovery (`wellness-core.recoveryScore`): HRV 0.5, RHR 0.2, sleep 0.15, resp 0.05, skin-temp 0.05, prior load 0.05, each z-scored against robust personal baselines; illness watch fires on 2-of-4 (HRV↓, RHR↑, resp↑, temp↑).

---

7. Physics-informed dynamics (`dynamics.py`) and other engines

- **Banister load→HRV** (impulse-response): fitness/fatigue from daily training impulse (TRIMP) with a temporal split and an honesty gate (needs ≥ 45 days with both load and HRV). `strain` from WHOOP days is stored as a signal to feed it.
- **Borbély two-process sleep pressure**: homeostatic Process S + circadian Process C.
- **Nutrition targets** (`nutrition.py`): Mifflin–St Jeor REE × measured PAL (from steps/strain), protein 1.6 g/kg (reference weight if BMI > 27), fibre 14 g/1000 kcal (≥ 30 g), WHO sugar/sat-fat/sodium caps, ICMR-NIN 45–55 % carbs; lab-aware notes. Meal analysis uses a curated Indian dish table + USDA FoodData Central SR Legacy (7 793 foods, CC0).
- **CGM** (`cgm.py`): International Consensus TIR metrics — mean, GMI (Bergenstal 2018), TIR 70–180, TBR, TAR, CV.
- **Twin** (`twin.py`): runs the person's routine/events through the above physiology to project risk and markers; lab projections every 10 days from real anchors, labelled "projected", scored when a new real report arrives.

---

8. AI stack — RAG, the model wrapper, guardrails, extensibility

Retrieval (RAG, `retrieval.py`): every data source becomes a small "card" (one lab marker, one 30-day signal, one insight, one tracker, nutrition targets, the WHOOP day, CGM summary, a memory, a cellular mechanism, biological age) — ~300+ cards. A question retrieves the relevant ones by **BM25** plus always-keep the flagged/critical cards. `connect()`

builds cross-domain **threads** (labs ↔ wearables ↔ genetics ↔ food lever), themed cardiovascular/metabolic/liver/sleep.

Model wrapper (`telomy_llm.py`): one provider-agnostic entry point. Priority: Anthropic (when `ANTHROPIC_API_KEY`) → any OpenAI-compatible endpoint (`TELOMY_LLM_BASE_URL`, covers free routes/OmniRoute) → none (deterministic composer).

Response cache + hard timeout. **Number-grounding guard:** every number in the answer must appear in the grounding context or the question, else the answer is rejected and the system falls back. Mirrored on-device in `slm.ts`.

Routing (the "smart engine"):

- *Quick + On-device* → Qwen3-1.7B on the phone (Cactus, int4), grounded on retrieved facts, guard-checked, escalates to the server on guard failure. App size unchanged (model fetched at runtime).
- *Quick + server* → composer, RAG-grounded.
- *Deep research* → RAG-first synthesis now (data + cross-domain connections + cellular mechanisms + literature, four sections), no cloud model needed; when a provider is configured the wrapper handles deep mode with a broader context.
- Future: Anthropic key → Deep upgrades to Claude automatically (pure config).

Cellular KB (`cellular.py`): evidence-graded (A–D) map of supplement/medicine/food/activity/stress → cell type → organelle/pathway → plain-language mechanism → the markers it moves (grounded in Reactome/WikiPathways/MitoCarta/NIH ODS). Feeds the RAG so Sinc can answer at the cellular level.

Science registry (`science.py`) — the engine's open edge (non-rigid): a new paper/guideline is added as a **row**, not code, and takes effect on the next request:

- `cross_panel_rule` — declarative conditions (marker op value / high / low / present, AND-ed) → fires a cited insight.
- `marker_range` — override an optimal range (a guideline tightens a target); feeds marker scoring and blood age.
- `cellular_effect` — add a mechanism to the cellular KB.

Each carries a citation + evidence grade, is append-only and reversible (`set_active`). Verified live: adding Tsimikas 2017 (Lp(a)) fired an insight instantly.

Talk-to-track (`trackers.py`): any statement to Sinc/voice becomes a tracker (pain NRS, symptom, supplement, medicine, habit, measurement, custom) with a baseline, a plan, red-flag safety routing (chest pain, stroke signs, self-harm → "get care now"), and a drivers analysis once ≥ 14 entries.

---

9. Multi-user design (per-user database)

Goal: each person signs in and sees only their own Vault, without a Postgres rewrite, mirroring the team's per-user isolation.

Approach — a database per user. `db.py` already opens a fresh connection per transaction from a path, so the only change is to make that path **per-request** via a context variable:

1. **Accounts store** (`accounts.py`): a shared `accounts.db` with `users(id, email, pw_hash, salt, created_at)` and `tokens(token, user_id, created_at)`. Passwords hashed with PBKDF2-HMAC-SHA256 (never stored plaintext).
2. **Per-user data DB**: `data/users/<user_id>.db`, created + schema-initialised on first login, seeded empty (no demo).
3. **Auth middleware**: resolve the bearer token → user_id → set `current_db` contextvar to that user's path for the

request. FastAPI copies the context into the threadpool, so every sync endpoint and `db.*` call uses the right file. Background threads (WHOOP warm loop, lifespan init) set the context explicitly.

4. **Endpoints**: `/auth/signup`, `/auth/login` (replace the single shared-password lock); everything else requires a valid user token. `/whoop/push` and `/nightscout/*` carry a per-user device token so a ring/CGM maps to its owner.

5. **Backward compatibility**: the existing single `telomy.db` migrates to the first user; with no accounts, a default single-user mode keeps working for local dev.

Isolation guarantee: no query joins across users because each user's data is a physically separate file — the strongest isolation, equivalent in effect to the team's Postgres row-level security, and simple to reason about, back up and delete (GDPR erase = delete one file).

Client: a login/sign-up screen gates the app; the token is kept in secure storage and sent on every request (the existing `unlock` flow is replaced by login). This is the remaining client piece after the backend lands.

---

10. Safety, privacy, verification

- **Never invent numbers** (grounding guard, both server and device). **Never fabricate genuine medical records** — synthetic data is always `is_test_data = 1` and labelled; the real record is preferred and kept.
- **Medicines** go to a clinician review queue; the app never changes doses. **Red-flag** phrases route to urgent care.
- **No secrets in code**; access lock (→ per-user auth in §9); device push tokens separate.
- **Verification**: 161 backend tests; `tools/audit.py` checks every app→route wiring (177/177) and sweeps every GET;

effect engine calibrated on a planted-truth cohort; models ship with held-out evidence and citations.

---

11. Devices & sensor strategy (single-device-per-user)

A user wears **one** device, so each must be standalone-complete. Common baseline in all three (PPG-HR, skin temp, IMU, flash+BLE+battery) + a hero per anatomy: **Patch** = chest ECG (HRV, AFib, respiration, best sleep staging); **Ring** = finger SpO■ + temp (best oximetry site, sleep, cycle); **Band** = wrist IMU (steps, activity, EDA, spot-ECG). Onboarding routes the user to the device matching their goal; all three feed the same model backend.

Full spec + body-site validation in `docs/research/DEVICE_SENSOR_SPEC.md`.

Recorder + calibration (2026-10): native iOS recorder (`telomy-lab/.../TelomyRec`) captures patch/ring/ oximeter/WHOOP on one clock → `/cal/*` backend + live monitor. Night-1 verdict (`docs/research/ CALIBRATION_NIGHT1.md`): the ECG patch quantified a +13–18 ms HRV inflation in PPG devices; **WHOOP SpO■/temp are NOT reconstructable from the NOOP raw stream** (1 Hz DC-only, no pulsatile AC) — drop WHOOP as a SpO■/temp source, calibrate the ring's finger SpO■ against a reference oximeter instead.

12. Roadmap (open)

Pace-of-aging + organ-specific aging clocks · vascular age (ECG→PPG PTT) · apnea/nocturnal-hypoxemia · HRV→autonomic age · VO■max · retrain AFib at scale · ring finger-SpO■ calibration (driven protocol) · activity/workout recogniser (PAMAP2/WISDM/Capture-24) · SQLite→Postgres for the multi-tenant plane · biobank-grade models (UK Biobank / MIMIC —

need access applications). Ranked with honest data-scale in `docs/research/TOP20_MODELS.md`; tracked in `docs/TASKS.md`.